

Contents

0.1	数据结构 update: 2026.4.28	2
0.2	前缀和/差分	2
0.3	树状数组	4
0.3.1	树状数组	4
0.3.2	维护区间加, 区间求和	5
0.3.3	二维树状数组	6
0.3.4	维护矩形区间加	7
0.4	线段树	8
0.4.1	维护区间最值	8
0.4.2	维护区间修改	10
0.4.3	维护最大子段和	12
0.4.4	动态开点	13
0.4.5	标记永久化	15
0.4.6	线段树上二分	15
0.4.7	维护树链	17
0.4.8	线段树合并	17
0.5	平衡树	23
0.5.1	红黑树	23
0.6	哈希表	25
0.7	主席树 (可持久化线段树)	26
0.7.1	维护值域	26
0.7.2	维护历史版本	29
0.7.3	维护树上信息	31
0.8	字典树	32
0.8.1	字典树	32
0.8.2	可持久化字典树	33
0.9	并查集	35
0.9.1	普通并查集	35
0.9.2	带权并查集	36
0.9.3	可撤销并查集	38
0.10	线性基	39
0.10.1	普通线性基	39
0.10.2	前缀线性基	42
0.11	ST 表	44
0.12	根号算法	46
0.12.1	莫队	46
0.13	堆	50
0.13.1	可删堆	50
0.13.2	对顶堆	51
0.13.3	可并堆 (pbds 实现)	53
0.14	其他	54
0.14.1	珂朵莉树	54
0.14.2	笛卡尔树	57

0.1 数据结构 update: 2026.4.28

0.2 前缀和/差分

```
{
    // 一维部分省略 ...
}

{
    // 用二次差分在区间  $[l, r]$  上加上等差数列: 首项  $s$ , 公差  $d$ 
    auto add_arithmetic = [&](vector<int>& d2, int l, int r, int s, int d) {
        if (l > r) return;
        d2[l] += s;
        if (l + 1 <= n + 1) d2[l + 1] += d - s;
        if (r + 1 <= n + 1) d2[r + 1] += -(s + (r - l + 1) * d);
        if (r + 2 <= n + 1) d2[r + 2] += s + (r - l) * d;
    };
}

{
    #include <iostream>
    #include <vector>

    using namespace std;

    // 定义我们要加的二次函数  $f(x) = ax^2 + bx + c$ 
    // 这里  $x$  是相对位置 (1, 2, 3...)
    long long f(long long x, long long a, long long b, long long c) {
        return a * x * x + b * x + c;
    }

    // 辅助函数: 计算在原数组绝对下标  $i$  处, 实际加上的函数值
    // 如果  $i$  小于起点  $l$ , 则无意义 (在我们的算法中不会被调用到非法位置)
    long long g(long long i, long long l, long long a, long long b, long long c) {
        long long x = i - l + 1; // 转换为相对位置
        return f(x, a, b, c);
    }

    // 三阶差分数组的区间修改
    void add_quadratic(vector<long long>& d3, int l, int r, long long a, long long b, long long c) {
        // 1. 起点  $l$  产生的启动变化 (+1 倍)
        d3[l] += g(1, l, a, b, c);
        d3[l + 1] += g(1 + 1, l, a, b, c) - 3 * g(1, l, a, b, c);
        d3[l + 2] += g(1 + 2, l, a, b, c) - 3 * g(1 + 1, l, a, b, c) + 3 * g(1, l, a, b, c);

        // 2. 终点  $r+1$  产生的抵消变化 (-1 倍)
        d3[r + 1] -= g(r + 1, l, a, b, c);
        d3[r + 2] -= g(r + 2, l, a, b, c) - 3 * g(r + 1, l, a, b, c);
        d3[r + 3] -= g(r + 3, l, a, b, c) - 3 * g(r + 2, l, a, b, c) + 3 * g(r + 1, l, a, b, c);
    }

    int main() {
        int n = 10; // 数组长度
        // 差分数组需要开大一点, 防止  $r+3$  越界。通常开  $N + 5$  即可
        vector<long long> d3(n + 5, 0);
        vector<long long> d2(n + 5, 0);
        vector<long long> d1(n + 5, 0);
        vector<long long> a_arr(n + 5, 0);
    }
}
```

```

// 示例: 在区间 [2, 5] 加上  $f(x) = x^2$  (a=1, b=0, c=0)
add_quadratic(d3, 2, 10, 1, 1, 0);

// 还原原数组: 做 3 次前缀和
for (int i = 1; i <= n; ++i) {
    d2[i] = d2[i - 1] + d3[i];      // 三阶 -> 二阶
    d1[i] = d1[i - 1] + d2[i];      // 二阶 -> 一阶
    a_arr[i] = a_arr[i - 1] + d1[i]; // 一阶 -> 原数组

    cout << a_arr[i] << " ";
}
// 输出预期: 0 1 4 9 16 0 0 0 0 0
cout << endl;

return 0;
}
}

// 三维前缀和 示例
int a,b,c;cin>>a>>b>>c;
vector pre(a+1,vector (b+1,vector<int> (c+1)));
for(int i = 1;i<=a;i++){
    for(int j = 1;j<=b;j++){
        for(int k = 1;k<=c;k++){
            cin>>pre[i][j][k];
        }
    }
}
for(int k = 1;k<=a;k++){
    for(int j = 1;j<=b;j++){
        for(int i = 1;i<=c;i++)
            pre[k][j][i] += pre[k][j][i-1];
        for(int i = 1;i<=c;i++)
            pre[k][j][i] += pre[k][j-1][i];
    }
    for(int j = 1;j<=b;j++){
        for(int i = 1;i<=c;i++)
            pre[k][j][i] += pre[k-1][j][i];
    }
}
cout<<pre[a][b][c]<<"\n";
}
{
    .....
}

```

0.3 树状数组

0.3.1 树状数组

```
template <typename T>
struct Fenwick {
    int n;
    std::vector<T> a;

    Fenwick(int n_ = 0) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        a.assign(n+5, T{});
    }

    void add(int x, const T &v) {
        for (int i = x; i <= n; i += i & -i) {
            a[i] = a[i] + v;
        }
    }

    T sum(int x) {
        T ans{};
        for (int i = x; i > 0; i -= i & -i) {
            ans = ans + a[i];
        }
        return ans;
    }

    T rangeSum(int l, int r) {
        return sum(r) - sum(l);
    }

    int select(const T &k) {
        int x = 0;
        T cur{};
        for (int i = 1 << std::__lg(n); i; i /= 2) {
            if (x + i <= n && cur + a[x + i] <= k) {
                x += i;
                cur = cur + a[x];
            }
        }
        return x;
    }
};
```

- select 函数返回 $\leq k$ 的最大索引 x , 复杂度 $\log(n)$

0.3.2 维护区间加, 区间求和

假设我们进行了一次操作: 给 $[1, x]$ 区间内的所有数加上 v 。

对于后续的查询 $[1, y]$, 这次更新产生的增量 Δ 分为两种情况:

- **情况 A:** $y \leq x$ 此时查询区间完全在更新区间内。增量为: $\Delta = y \cdot v$ 。
- **情况 B:** $y > x$ 此时查询区间覆盖了整个更新区间。增量为: $\Delta = x \cdot v$ 。

用两个树状数组 A 和 B 分别维护了这两种情况的贡献:

- **树状数组 B :** 维护的是“单位增量 v ”, 通过后缀和 $B[y+1, n]$ 捕获那些 $y \leq x$ 的更新。
- **树状数组 A :** 维护的是“封顶增量 $x \cdot v$ ”, 通过前缀和 $A[1, y]$ 捕获那些 $y > x$ 的更新。

```
int n,m;cin>>n>>m;
vector<int> a(n+1);
for(int i = 1;i<=n;i++){
    cin>>a[i];a[i] += a[i-1];
}
BIT<int> A(n+10),B(n+10);
for(int i = 1;i<=m;i++){
    int o;cin>>o;
    if(o == 1){
        int x,y,k;cin>>x>>y>>k;
        if(x>y) swap(x,y);
        A.add(y, y*k);B.add(y, k);
        A.add(x-1, -(x-1)*k);B.add(x-1, -k);
    }else{
        int x,y;cin>>x>>y;
        if(x>y) swap(x,y);
        int res1 = A.getsum(y)+(B.getsum(n)-B.getsum(y))*y;
        int res2 = A.getsum((x-1))+(B.getsum(n)-B.getsum((x-1)))*(x-1);
        cout<<res1-res2+a[y]-a[x-1]<<"\n";
    }
}
```

0.3.3 二维树状数组

```
template <class T>
struct BIT
{
    int n, m;
    std::vector<std::vector<T>> a;
    int lowbit(int x)
    {
        return x & (-x);
    }
    BIT(int n, int m) : n(n), m(m)
    {
        a.assign(n + 1, std::vector<T>(m + 1, 0));
    }
    BIT(std::vector<std::vector<T>> &val) : n(val.size() - 1), m(val[1].size() - 1)
    {
        a.assign(n + 1, std::vector<int>(m + 1, 0));
        for (int i = 1; i <= n; ++i)
        {
            for (int j = 1; j <= m; ++j)
            {
                add(i, j, a[i][j]);
            }
        }
    }

    void add(int x, int y, T val)
    {
        for (int i = x; i <= n; i += lowbit(i))
        {
            for (int j = y; j <= m; j += lowbit(j))
            {
                a[i][j] += val;
            }
        }
    }

    T query(int x, int y)
    {
        T res = 0;
        for (int i = x; i >= 1; i -= lowbit(i))
        {
            for (int j = y; j >= 1; j -= lowbit(j))
            {
                res += a[i][j];
            }
        }
        return res;
    }

    T query(int x1, int y1, int x2, int y2)
    {
        return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
    }
};
```

0.3.4 维护矩形区间加

原理和一维的类似,(P4514上帝造题的七分钟)

```
template <class T>
struct Bit
{
    int n, m;
    BIT<T> A, Ai, Aj, Aij;
    Bit(int n, int m) : n(n), m(m), A(n, m), Ai(n, m), Aj(n, m), Aij(n, m) // 只用空构造
    {
    }

    void add(int x, int y, T val)
    {
        A.add(x, y, val);
        Ai.add(x, y, x * val);
        Aj.add(x, y, y * val);
        Aij.add(x, y, x * y * val);
    }

    void add(int x1, int y1, int x2, int y2, T val)
    {
        add(x1, y1, val);
        add(x1, y2 + 1, -val);
        add(x2 + 1, y1, -val);
        add(x2 + 1, y2 + 1, val);
    }

    T query(int x, int y)
    {
        return ((T)x * y + x + y + 1) * A.query(x, y) - Ai.query(x, y) * (y + 1) - Aj.query(x, y) * (x + 1) + Aij.query(x, y);
    }

    T query(int x1, int y1, int x2, int y2)
    {
        return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
    }
};
```

0.4 线段树

0.4.1 维护区间最值

```
template <class T>
class RMQSegmentTree
{
#define lc u << 1
#define rc u << 1 | 1
    const T inf = std::numeric_limits<T>::max() / 2;

public:
    struct Node
    {
        int l, r;
        T max, min;
    };
    RMQSegmentTree(const std::vector<T> &a) : n(a.size()), tr(4 * n)
    {
        n--;
        std::function<void(int, int, int)> build = [&](int u, int l, int r)
        {
            tr[u] = {l, r, a[l], a[l]};
            if (l == r)
            {
                return;
            }
            int mid = (l + r) >> 1;
            build(lc, l, mid);
            build(rc, mid + 1, r);
            pushup(u);
        };
        build(1, 1, n);
    }
    void singleChange(int pos, T val)
    {
        singleChange(1, 1, n, pos, val);
    }
    T rangeQueryMax(int l, int r)
    {
        return rangeQueryMax(1, 1, n, l, r);
    }
    T rangeQueryMin(int l, int r)
    {
        return rangeQueryMin(1, 1, n, l, r);
    }

private:
    int n;
    std::vector<Node> tr;
    void pushup(int u)
    {
        tr[u].max = std::max(tr[lc].max, tr[rc].max);
        tr[u].min = std::min(tr[lc].min, tr[rc].min);
    }

    void singleChange(int u, int l, int r, int x, T y)
```



```

{
    if (l == r)
    {
        tr[u].max = tr[u].min = y;
        return;
    }
    int mid = (l + r) >> 1;
    if (x <= mid)
    {
        singleChange(lc, l, mid, x, y);
    }
    else
    {
        singleChange(rc, mid + 1, r, x, y);
    }
    pushup(u);
}

T rangeQueryMax(int u, int l, int r, int x, int y)
{
    if (x <= l && y >= r)
    {
        return tr[u].max;
    }
    int mid = (l + r) >> 1;
    T res = -inf;
    if (x <= mid)
    {
        res = std::max(res, rangeQueryMax(lc, l, mid, x, y));
    }
    if (y > mid)
    {
        res = std::max(res, rangeQueryMax(rc, mid + 1, r, x, y));
    }
    return res;
}

T rangeQueryMin(int u, int l, int r, int x, int y)
{
    if (x <= l && y >= r)
    {
        return tr[u].min;
    }
    int mid = (l + r) >> 1;
    T res = inf;
    if (x <= mid)
    {
        res = std::min(res, rangeQueryMin(lc, l, mid, x, y));
    }
    if (y > mid)
    {
        res = std::min(res, rangeQueryMin(rc, mid + 1, r, x, y));
    }
    return res;
}
};

```

0.4.2 维护区间修改

```
template <class T>
class SegmentTree
{
#define lc u << 1
#define rc u << 1 | 1
public:
    struct Node
    {
        int l, r;
        T add, sum;
    };

    SegmentTree(const std::vector<T> &a) : n(a.size()), tr(4 * n)
    {
        n--;
        std::function<void(int, int, int)> build = [&](int u, int l, int r)
        {
            tr[u] = {l, r, 0, a[l]};
            if (l == r)
            {
                return;
            }
            int mid = (l + r) >> 1;
            build(lc, l, mid);
            build(rc, mid + 1, r);
            pushup(u);
        };
        build(1, 1, n);
    }
    void rangeAdd(int l, int r, T val)
    {
        rangeAdd(1, 1, n, l, r, val);
    }
    T rangeQuerySum(int l, int r)
    {
        return rangeQuerySum(1, 1, n, l, r);
    }

private:
    int n;
    std::vector<Node> tr;
    void pushup(int u)
    {
        tr[u].sum = tr[lc].sum + tr[rc].sum;
    }
    void pushdown(Node &u, T add)
    {
        u.sum += add * (u.r - u.l + 1);
        u.add += add;
    }
    void pushdown(int u)
    {
        if (tr[u].add)
        {
            pushdown(tr[lc], tr[u].add);
            pushdown(tr[rc], tr[u].add);
        }
    }
}
```

```

        pushdown(tr[rc], tr[u].add);
        tr[u].add = 0;
    }
}

void rangeAdd(int u, int l, int r, int x, int y, T k)
{
    if (x <= l && y >= r)
    {
        pushdown(tr[u], k);
        return;
    }
    int mid = (l + r) >> 1;
    pushdown(u);
    if (x <= mid)
    {
        rangeAdd(lc, l, mid, x, y, k);
    }
    if (y > mid)
    {
        rangeAdd(rc, mid + 1, r, x, y, k);
    }
    pushup(u);
}

T rangeQuerySum(int u, int l, int r, int x, int y)
{
    if (x <= l && y >= r)
    {
        return tr[u].sum;
    }
    int mid = (l + r) >> 1;
    pushdown(u);
    T res = 0;
    if (x <= mid)
    {
        res += rangeQuerySum(lc, l, mid, x, y);
    }
    if (y > mid)
    {
        res += rangeQuerySum(rc, mid + 1, r, x, y);
    }
    return res;
}
};

```

乘标记和加标记的处理方法：

区间乘和区间加两个操作，关键在于维护乘法标记和加法标记之间的关系

具体地说，我们在下传标记时，采用的计算和的方法为 $sum = sum * mul + add * len$ ，那么我们就保证下传标记的时候正确操作，即 $u.add = u.add * mul + add$, $u.mul = u.mul * mul$

覆盖标记和加标记的处理方法：

区间覆盖和区间加，关键在于懒标记之间的清除关系，在我们操作 *cover* 标记时，要清空对应位置的 *add* 标记，这样才能保证信息维护的正确性。具体来讲，什么叫清空对应位置的 *add* 标记呢？也就是说，一旦我们更新了 *u* 位置的 *cover* 信息，我们就要立刻把 *u* 位置的 *add* 信息清除掉，因为显然他不会再起作用，而且在 *pushdown* 的时候我们要先处理 *cover* 标记，原因仍然是我们可能会清除 *add* 标记

0.4.3 维护最大子段和

支持单点修改和区间查询最大子段和，不支持区间修改操作。

```
template <class T>
class SegmentTree{
#define lc u << 1
#define rc u << 1 | 1
public:
    struct Node{
        int l, r;
        T sum, lmax, rmax, max;
    };
    SegmentTree(const std::vector<T> &a) : n(a.size()), tr(n * 4) {
        n--;
        std::function<void(int, int, int)> build = [&](int u, int l, int r){
            tr[u] = {l, r, a[l], a[l], a[l], a[l]};
            if (l == r){
                return;
            }
            int mid = l + r >> 1;
            build(lc, l, mid);
            build(rc, mid + 1, r);
            pushup(tr[u], tr[lc], tr[rc]);
        };
        build(1, 1, n);
    }
    T rangeQuery(int x, int y){
        Node ans = rangeQuery(1, 1, n, x, y);
        return ans.max;
    }
private:
    int n;
    std::vector<Node> tr;
    void pushup(Node &u, Node l, Node r){
        u.sum = l.sum + r.sum;
        u.lmax = std::max(l.lmax, l.sum + r.lmax);
        u.rmax = std::max(r.rmax, r.sum + l.rmax);
        u.max = std::max({l.max, r.max, l.rmax + r.lmax});
    }
    Node rangeQuery(int u, int l, int r, int x, int y) {
        if (x <= l && y >= r){
            return tr[u];
        }
        int mid = l + r >> 1;
        if (y <= mid){
            return rangeQuery(lc, l, mid, x, y);
        }
        if (x > mid){
            return rangeQuery(rc, mid + 1, r, x, y);
        }
        Node t;
        pushup(t, rangeQuery(lc, l, mid, x, y), rangeQuery(rc, mid + 1, r, x, y));
        return t;
    }
};
```

0.4.4 动态开点

往往我们的序列非常大，但大多数为空，我们并不需要维护整个序列的信息，因此我们可以采用动态开空间的方法，于是有动态开点线段树，具体的说，就是用到哪里的空间就开哪里的空间，我们每需要访问一个节点 u ，假如节点 u 尚未分配空间，我们手动为其开空间即可，因为操作次数往往有限，每次操作我们最多开 $\log n$ 的空间，所以空间复杂度是 $q \log n$ 的，也就是传的 `capacity`

```
template <class T>
class SegmentTree
{
public:
    SegmentTree(int capacity, int min, int max) : max(max), min(min)
    {
        lc.assign(capacity + 1, 0);
        rc.assign(capacity + 1, 0);
        sum.assign(capacity + 1, 0);
        add.assign(capacity + 1, 0);
    }

    void rangeAdd(int l, int r, T val)
    {
        rangeAdd(root, min, max, l, r, val);
    }
    T rangeQuerySum(int l, int r)
    {
        return rangeQuerySum(root, min, max, l, r);
    }

private:
    int min,
        max;
    int tot = 0;
    int root = 0;
    std::vector<int> lc, rc;
    std::vector<T> sum, add;
    void pushup(int u)
    {
        sum[u] = sum[lc[u]] + sum[rc[u]];
    }
    void pushdown(int u, int l, int r)
    {
        if (add[u])
        {
            lc[u] = (lc[u] ? lc[u] : ++tot);
            rc[u] = (rc[u] ? rc[u] : ++tot);
            int mid = (l + r) >> 1;
            sum[lc[u]] += (mid - l + 1) * add[u];
            sum[rc[u]] += (r - mid) * add[u];
            add[lc[u]] += add[u];
            add[rc[u]] += add[u];
            add[u] = 0;
        }
    }
    void rangeAdd(int &u, int l, int r, int x, int y, T val)
    {
        // 区修
        if (!u)
        {
            u = ++tot; // 动态开点
        }
    }
};
```

```

    }
    if (x <= l && r <= y)
    {
        sum[u] += val * (r - l + 1);
        add[u] += val;
        return;
    }
    pushdown(u, l, r);
    int mid = (l + r) >> 1;
    if (x <= mid)
    {
        rangeAdd(lc[u], l, mid, x, y, val);
    }
    if (y > mid)
    {
        rangeAdd(rc[u], mid + 1, r, x, y, val);
    }
    pushup(u);
}
T rangeQuerySum(int u, int l, int r, int x, int y)
{
    if (x <= l and y >= r)
    {
        return (u ? sum[u] : 0);
    }
    int mid = (l + r) >> 1;
    pushdown(u, l, r);
    T res = 0;
    if (x <= mid)
    {
        res += rangeQuerySum(lc[u], l, mid, x, y);
    }
    if (y > mid)
    {
        res += rangeQuerySum(rc[u], mid + 1, r, x, y);
    }
    return res;
}
};

```

0.4.5 标记永久化

标记永久化是另一种处理懒标记的技术，我们可以把标记永久打在某个区间而不下传给子区间，我们只需要在查询时累加经过节点的信息即可。

0.4.6 线段树上二分

线段树上二分，其作用往往是找到 $[L, R]$ 区间内第一个大于等于 D 的位置或值，同理也可以求出右边

我们常规的想法是在线段树外二分并不断询问区间 max ，但这样的复杂度是 $(\log n)^2$ 的，并不够优秀

我们完全可以直接在线段树上进行这个二分操作，复杂度降为 $\log(n)$

```
template <class T>
class SegmentTree
{
#define lc u << 1
#define rc u << 1 | 1

public:
    struct Node
    {
        int l, r;
        T max;
    };
    SegmentTree(const std::vector<T> &a) : n(a.size()), tr(4 * n)
    {
        n--;
        std::function<void(int, int, int)> build = [&](int u, int l, int r)
        {
            tr[u] = {l, r, a[l]};
            if (l == r)
            {
                return;
            }
            int mid = l + r >> 1;
            build(lc, l, mid);
            build(rc, mid + 1, r);
            pushup(u);
        };
        build(1, 1, n);
    }
    int queryLeftFirstlower(int x, int y, T d)
    {
        return queryLeftFirstlower(1, 1, n, x, y, d);
    }

private:
    void pushup(int u)
    {
        tr[u].max = std::max(tr[lc].max, tr[rc].max);
    }
    int n;
    std::vector<Node> tr;
    int queryLeftFirstlower(int u, int l, int r, int x, int y, T d) // [x, y] 靠左的第一个  $\geq d$  的下标
    {
        if (x == l and y == r)
        {
            if (tr[u].max < d)
            {
```

```

        return 0;
    }
    if (l == r)
    {
        return 1;
    }
    int mid = l + r >> 1;
    if (tr[lc].max >= d)
    {
        return queryLeftFirstlower(lc, l, mid, x, mid, d);
    }
    else
    {
        return queryLeftFirstlower(rc, mid + 1, r, mid + 1, y, d);
    }
}
else
{
    int mid = l + r >> 1;
    if (y <= mid)
    {
        return queryLeftFirstlower(lc, l, mid, x, y, d);
    }
    else if (x > mid)
    {
        return queryLeftFirstlower(rc, mid + 1, r, x, y, d);
    }
    else
    {
        int pos = queryLeftFirstlower(lc, l, mid, x, mid, d);
        if (pos == 0)
        {
            return queryLeftFirstlower(rc, mid + 1, r, mid + 1, y, d);
        }
        else
        {
            return pos;
        }
    }
}
}
};

```


0.4.7 维护树链

树链剖分得到 dfn 序列和重链信息，可以把链转成 $\log(n)$ 个区间，于是我们可以用线段树维护树上链的信息

0.4.8 线段树合并

线段树合并往往是动态开点的权值线段树，往往解决的是关于值域方面的询问，比如问子树内出现最多的颜色

换个理解方法，也就是建在树上的主席树

询问往往分两种常见形式

其一是每个节点有信息，询问以某点的子树内的信息总和，这一类是朴素的线段树合并

其二是区间链加的操作，询问单点信息，这一类我们转化为树上差分，即可转化为第一类。

对于求解方法，我们仍然分两类，新开节点和不新开节点。

对于不新开节点的做法，我们往往离线询问到对应的节点上，然后在递归合并 dfs 时，当 u 节点信息合并完成，我们顺势处理 u 节点的所有询问，但是之后该节点的信息会被覆盖而不能再使用

对于新开节点的做法，我们可以直接 dfs 递归合并所有信息，这样每个节点都可以保留正确的信息，我们可以不用离线询问，但是空间消耗巨大。

往往我们直接采用第一种做法即可

Solution1

```
class MergeSegmentTree
{
public:
    // 构造函数
    // g: 树的邻接表表示
    // val: 各节点的值
    // capacity: 线段树节点预分配空间
    // treeRoot: 树的根节点
    // min/max: 值域范围 (若值域过大需先离散化)
    MergeSegmentTree(const std::vector<std::vector<int>> &g, std::vector<int> &val, int capacity, int treeRoot,
                     int min, int max) : n(val.size()), g(g), min(min), max(max)
    {
        --n; // 调整为 1-based 索引
        // 初始化线段树节点数组
        root.assign(capacity + 1, 0); // 各子树根节点
        lc.assign(capacity + 1, 0);   // 左子节点指针
        rc.assign(capacity + 1, 0);   // 右子节点指针
        sum.assign(capacity + 1, 0);  // 区间和

        // 为每个节点创建叶子节点
        for (int i = 1; i <= n; ++i)
        {
            insert(root[i], min, max, val[i], 1); // 插入节点值
        }
        // 后序遍历合并子树线段树
        dfs(treeRoot, 0);
    }

    // 查询 u 子树中值 >=val 的节点数量
    int query(int u, int val)
    {
        return query(root[u], min, max, val);
    }
};
```

```

private:
    int n; // 节点总数
    int tot = 0; // 线段树节点计数器
    int min, max; // 值域范围
    std::vector<int> root; // 各子树根节点
    std::vector<int> lc; // 左子节点指针
    std::vector<int> rc; // 右子节点指针
    std::vector<int> sum; // 区间和统计
    std::vector<std::vector<int>> g; // 树的邻接表

    // 更新节点统计值
    void pushup(int u)
    {
        sum[u] = sum[lc[u]] + sum[rc[u]];
    }

    // 在线段树中插入值
    // u: 当前节点指针 (引用传递)
    // l/r: 当前区间
    // pos: 插入位置
    // val: 插入值
    void insert(int &u, int l, int r, int pos, int val)
    {
        if (!u)
            u = ++tot; // 动态开点
        if (l == r)
        {
            sum[u] += val; // 叶子节点更新
            return;
        }
        int mid = l + r >> 1;
        if (pos <= mid)
            insert(lc[u], l, mid, pos, val); // 递归左子树
        else
            insert(rc[u], mid + 1, r, pos, val); // 递归右子树
        pushup(u); // 更新统计值
    }

    // 合并两棵线段树 (返回合并后的根节点)
    int merge(int x, int y)
    {
        if (!x)
            return y; // x 为空则返回 y
        if (!y)
            return x; // y 为空则返回 x
        // 创建新节点合并统计值
        int u = ++tot;
        sum[u] = sum[x] + sum[y];
        // 递归合并左右子树
        lc[u] = merge(lc[x], lc[y]);
        rc[u] = merge(rc[x], rc[y]);
        return u;
    }

    // 后序遍历合并子树
    void dfs(int u, int fa)

```

```

{
    for (auto v : g[u])
    {
        if (v == fa)
            continue; // 跳过父节点
        dfs(v, u); // 先处理子节点
        root[u] = merge(root[u], root[v]); // 合并子节点线段树
    }
}

// 查询区间中 >=val 的值的数量
int query(int u, int l, int r, int val)
{
    if (!u)
        return 0; // 空树返回 0
    if (l >= val)
        return sum[u]; // 整个区间都 >=val
    int res = 0;
    int mid = l + r >> 1;
    if (mid >= val)
        res += query(lc[u], l, mid, val); // 左子树可能包含
    res += query(rc[u], mid + 1, r, val); // 右子树可能包含
    return res;
}
};

```

Solution2

```
class MergeSegmentTree
{
public:
    // 构造函数
    // g: 树的邻接表表示
    // val: 每个节点的值
    // capacity: 线段树的最大容量
    // treeRoot: 树的根节点
    // min: 值域的最小值
    // max: 值域的最大值 (如果值域太大可以传入离散化后的数组)
    MergeSegmentTree(const std::vector<std::vector<int>> &g, std::vector<int> &val, int capacity, int treeRoot, int n) : n(val.size()), ans(n), g(g), min(min), max(max)
    {
        --n; // 调整节点数量 (假设节点编号从 1 开始)
        // 初始化线段树的各个数组
        root.assign(capacity + 1, 0); // 每个节点的线段树根
        lc.assign(capacity + 1, 0); // 左孩子数组
        rc.assign(capacity + 1, 0); // 右孩子数组
        sum.assign(capacity + 1, 0); // 线段树节点对应的区间和

        // 为每个节点创建初始线段树
        for (int i = 1; i <= n; ++i)
        {
            insert(root[i], min, max, val[i], 1); // 在值 val[i] 位置插入 1
        }

        // 从树的根节点开始进行深度优先遍历
        dfs(treeRoot, 0);
    }

    // 获取结果数组
    std::vector<int> getAns()
    {
        return ans;
    }

private:
    int n; // 节点数量
    int tot = 0; // 线段树节点计数器
    int min, max; // 值域范围
    std::vector<int> root; // 每个节点的线段树根
    std::vector<int> lc; // 线段树左孩子
    std::vector<int> rc; // 线段树右孩子
    std::vector<int> sum; // 线段树节点和
    std::vector<std::vector<int>> g; // 树的邻接表
    std::vector<int> ans; // 存储每个节点的答案

    // 更新线段树节点的和
    void pushup(int u)
    {
        sum[u] = sum[lc[u]] + sum[rc[u]];
    }

    // 在线段树中插入值
    // u: 当前线段树节点
```

```

// l, r: 当前区间
// pos: 要插入的位置
// val: 要插入的值
void insert(int &u, int l, int r, int pos, int val)
{
    if (!u)
        u = ++tot; // 动态开点
    if (l == r)
    {
        sum[u] += val; // 叶子节点更新
        return;
    }
    int mid = l + r >> 1;
    if (pos <= mid)
    {
        insert(lc[u], l, mid, pos, val); // 递归左子树
    }
    else
    {
        insert(rc[u], mid + 1, r, pos, val); // 递归右子树
    }
    pushup(u); // 更新当前节点
}

// 合并两棵线段树 (覆盖式合并)
int merge(int x, int y)
{
    if (!x)
        return y; // 如果 x 为空, 返回 y
    if (!y)
        return x; // 如果 y 为空, 返回 x
    sum[x] += sum[y]; // 合并节点和
    lc[x] = merge(lc[x], lc[y]); // 递归合并左子树
    rc[x] = merge(rc[x], rc[y]); // 递归合并右子树
    return x;
}

// 深度优先遍历树
void dfs(int u, int fa)
{
    for (auto v : g[u]) // 遍历所有子节点
    {
        if (v == fa)
            continue; // 跳过父节点
        dfs(v, u); // 递归处理子节点
        root[u] = merge(root[u], root[v]); // 将子节点的线段树合并到当前节点
    }
    ans[u] = query(root[u], min, max, val); // 查询当前节点的答案
}

// 查询线段树中 >=val 的值的数量
int query(int u, int l, int r, int val)
{
    if (!u)
        return 0; // 空树返回 0
    if (l >= val)

```

```

        return sum[u]; // 整个区间都 >=val, 直接返回和
    int res = 0;
    int mid = l + r >> 1;
    if (mid >= val)
    {
        res += query(lc[u], l, mid, val);
    }
    res += query(rc[u], mid + 1, r, val);
    return res;
}
};

```

0.5 平衡树

平衡树是一种 BST，也就是二叉搜索树。在理想状态下我们的操作复杂度为 $O(\log(n))$ ，但是在退化为链为复杂度将退化为 $O(\log(n))$ ，因此我们往往通过某些手段来尽可能维护树的平衡性，因此衍生出了不同种类的平衡树

目前我们主流的平衡树有两种，其一是 FHQ-Treap，其二是 Splay，其中 FHQ 支持可持久化的操作，Splay 在维护 LCT 的时候复杂度更优，各自具有其优势。

平衡树同样具有两种用法，维护区间和维护值域，类似于区间线段树和权值线段树

相较于线段树而言，平衡树最大的功能是可以某些位置进行插入删除操作并同时维护序列的信息，而线段树只具有修改的功能，但不能在序列的中间位置插入和删除。

由于 FHQ 更容易理解且更好写，我们往往只在考虑 LCT 时使用 Splay。

0.5.1 红黑树

在 pbds 库中有内置的红黑树，我们传 `std::less` 即为 `set`，想要实现可重集合有两种方法，其一是传入 `std::less_equal`，其二是传入 `std::less`，第二维是一个全局自增变量，也就是通过第二维使得相同的数字被判定为不同。

```
#include<bits/extc++.h>
// #include <ext/pb_ds/assoc_container.hpp>
// #include <ext/pb_ds/tree_policy.hpp>

namespace pb = __gnu_pbds;
template <class T, class Cmp = std::less<T>>
// Cmp 传 std::less<T> 就是 Set 传 std::less_equal<T> 就是 multiset
struct RedBlackTree
{
    // order_of_key(x) 返回严格小于 x 的元素个数
    // find_by_order(x) 返回从小到大第 x+1 个数的迭代器，从 0 开始计数
    // split(val,t) 把 >=val 的部分给 t
    const T inf = std::numeric_limits<T>::max() / 2;
    using Tree = pb::tree<T, pb::null_type, Cmp, pb::rb_tree_tag, pb::tree_order_statistics_node_update>;
    Tree tree;
    RedBlackTree() {}
    void insert(T x)
    {
        tree.insert(x);
    }
    void eraseOne(T x)
    {
        auto it = tree.find_by_order(tree.order_of_key(x)); // 更可靠的方式找到第一个 x
        if (it != tree.end() and *it == x)
        {
            tree.erase(it);
        }
    }
    void eraseAll(T x)
    {
        Tree a, b;
        tree.split(x, a); // 把
        a.split(x + 1, b);
        tree.join(b);
    }
    int getNum(T x)
    {
        return tree.order_of_key(x + 1) - tree.order_of_key(x);
    }
    int allNum()
    {

```

```

{
    return tree.size();
}
T nxt(T x)
{
    insert(x + 1);
    auto it = tree.find_by_order(tree.order_of_key(x + 1));
    ++it;
    if (it == tree.end())
    {
        return inf;
    }
    eraseOne(x + 1);
    return *it;
}
T pre(T x)
{
    auto it = tree.find_by_order(tree.order_of_key(x));
    if (it == tree.begin())
    {
        return -inf;
    }
    --it;
    return *it;
}
// 3 2 2 1 1
// 从大到小排
int getRank(T x) // 默认最大的为 rank1 相同元素我最大
{
    return tree.size() - tree.order_of_key(x + 1) + 1;
}
T getVal(int rank)
{
    return *tree.find_by_order(tree.size() - rank);
}
// 从小到大排
int getAntiRank(T x)
{
    return tree.order_of_key(x) + 1;
}
T getAntiVal(int rank)
{
    return *tree.find_by_order(rank - 1);
}
};

```


0.6 哈希表

考虑使用安全的 `gp_hash_table` 来代替 `umap` , 比 `umap` 快很多

如果传入的 `Key` 不被直接支持就利用 `custom_hash` 把所有元素糅合一下

不考虑顺序的情况下完全可以代替 `map` 和 `umap`, 只是占的空间可能有一点大

```
#include<bits/stdc++.h>
#include<bits/extc++.h>
// 包含 pbds 库 (部分编译器可能需要用 #include <ext/pb_ds/hash_policy.hpp>)
// #include <ext/pb_ds/assoc_container.hpp>
// #include <ext/pb_ds/hash_policy.hpp>

// using namespace __gnu_pbds; 可选

#define ull unsigned long long
#define u64 uint64_t;
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        // 保证输入分布均匀
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        // 使用 steady_clock 或地址作为种子, 防止固定数据 Hack
        // 由于 static 不用担心传入同一个 x 会返回不同的值
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
        // 核心: 将输入先与随机盐异或, 再跑一遍 SplitMix64
        return splitmix64(x ^ FIXED_RANDOM);
    }
};
// 使用方法
// __gnu_pbds::gp_hash_table<Key, Value, custom_hash> mp;
// 当 map, umap 用就可以
```

0.7 主席树（可持久化线段树）

静态区间第 k 小

本质就是动态开点的（如果不离散化）可持久化权值线段树

操作是注意插入元素的值域，需要满足在 $[1, \text{inf}]$ 到范围内，如果出现负数考虑：

- 离散化（值域很大也要离散化，大一点点无所谓）
- 把所有数都加 $\text{inf}+1$ 使得值域变成 $[1, 2*\text{inf}+1]$

!!!（注意不要直接加 $1e9$ 再 $-1e9$ ）尤其是最后输出整数结果的时候 $-1e9$ ，会自动转化为浮点数操作最后，比如像下面这样

```
cout<<1e9<<"\n"<<666-1e9<<"\n";
// 输出
1e+09
-9.99999e+08
```

0.7.1 维护值域

```
template <class T>
class PresidentTree
{
private:
    // 线段树节点结构体
    struct Node
    {
        int val; // 节点值（通常是区间统计值）
        int l, r; // 左右子节点索引
        T sum;
        // 构造函数，默认左右子节点为 0（空）
        Node(int val = 0) : val{val}, l{0}, r{0}, sum{0} {}
    };

    std::vector<Node> tr; // 动态节点存储数组
    const int Start, Last; // 值域范围 [Start, Last]
    std::vector<int> root; // 各版本根节点索引数组
    int newNode(int val = 0) // 创建新节点
    {
        tr.emplace_back(Node(val)); // 在数组末尾添加新节点
        return tr.size() - 1; // 返回新节点索引
    }

    // 向上更新节点值（基于子节点）
    void pushup(int u)
    {
        tr[u].val = tr[tr[u].l].val + tr[tr[u].r].val; // 当前节点值为左右子节点值之和
        tr[u].sum = tr[tr[u].l].sum + tr[tr[u].r].sum;
    }

    // 插入操作（动态开点）
    void insert(int &u, int l, int r, int x)
    {
        if (u == 0) // 如果当前节点不存在
        {
            u = newNode(); // 创建新节点
        }
        if (l == r) // 到达叶子节点
        {
```

```

        tr[u].val++; // 增加计数 (用于统计出现次数)
        tr[u].sum += 1;
        return;
    }
    int mid = (l + r) >> 1; // 计算中点
    if (x <= mid) // 目标在左子树
    {
        insert(tr[u].l, l, mid, x); // 递归处理左子树
    }
    else // 目标在右子树
    {
        insert(tr[u].r, mid + 1, r, x); // 递归处理右子树
    }
    pushup(u); // 更新当前节点值
}

// 合并两棵线段树 (用于构建版本链)
int merge(int u, int v, int l, int r)
{
    if (!u or !v) // 如果任一节点为空
    {
        return (u ? u : v); // 返回非空的那个
    }
    if (l == r) // 到达叶子节点
    {
        tr[u].val += tr[v].val; // 合并统计值
        tr[u].sum += tr[v].sum;
        return u;
    }
    int mid = (l + r) >> 1; // 计算中点
    tr[u].l = merge(tr[u].l, tr[v].l, l, mid); // 递归合并左子树
    tr[u].r = merge(tr[u].r, tr[v].r, mid + 1, r); // 递归合并右子树
    pushup(u); // 更新当前节点值
    return u; // 返回合并后的树根
}

// 查询区间 [x,y] 内的元素个数 (版本 u 到 v 之间的变化)
int getRange(int u, int v, int l, int r, int x, int y)
{
    if (y < l or x > r) // 查询区间与当前区间无交集
    {
        return 0;
    }
    if (x <= l and y >= r) // 当前区间完全包含在查询区间内
    {
        return tr[v].val - tr[u].val; // 返回版本间的差值
    }
    int mid = (l + r) >> 1; // 计算中点
    int res = 0;
    if (x <= mid) // 查询左子树
    {
        res += getRange(tr[u].l, tr[v].l, l, mid, x, y);
    }
    if (y > mid) // 查询右子树
    {
        res += getRange(tr[u].r, tr[v].r, mid + 1, r, x, y);
    }
}

```

```

    }
    return res;
}

// 查询第 k 小的元素 (版本 u 到 v 之间)
int getKth(int u, int v, int l, int r, int k)
{
    if (l == r) // 到达叶子节点
    {
        return l; // 返回该值
    }
    int mid = (l + r) >> 1; // 计算中点
    int L = tr[tr[v].l].val - tr[tr[u].l].val; // 左子树元素个数
    if (L >= k) // 第 k 小在左子树
    {
        return getKth(tr[u].l, tr[v].l, l, mid, k);
    }
    else // 第 k 小在右子树
    {
        return getKth(tr[u].r, tr[v].r, mid + 1, r, k - L); // 注意 k 要减去左子树元素数
    }
}

// 前 k 大的数之和
T getKthSum(int u, int v, int l, int r, int k)
{
    if (!k)
    {
        return 0;
    }
    if (l == r)
    {
        return (T)l * k;
    }
    int cnt = tr[tr[v].r].val - tr[tr[u].r].val;
    int mid = (l + r) >> 1;
    if (cnt >= k)
    {
        return getKthSum(tr[u].r, tr[v].r, mid + 1, r, k);
    }
    else
    {
        return tr[tr[v].r].sum - tr[tr[u].r].sum + getKthSum(tr[u].l, tr[v].l, l, mid, k - cnt);
    }
}
}

```

public:

```

// 构造函数: 基于数组 a 初始化, 值域范围 [mi, ma]
PresidentTree(const std::vector<int> &a, int mi, int ma)
: root(a.size()), Start(mi), Last(ma), tr(1) // 初始化
{
    // 预分配空间 (优化性能)
    tr.reserve(a.size() * std::__lg(2 * a.size()));
    root[0] = newNode(); // 创建初始版本 (空树)

    // 构建各版本
    for (int i = 1; i <= a.size() - 1; ++i)

```

```

{
    // 空间不足时扩容
    if (tr.capacity() <= tr.size() + 64)
    {
        tr.reserve(std::max(2 * tr.capacity(), tr.capacity() + 64));
    }
    insert(root[i], Start, Last, a[i]); // 插入当前元素
    root[i] = merge(root[i], root[i - 1], Start, Last); // 合并到前一版本
}

// 查询区间  $[u, v]$  中值在  $[l, r]$  范围内的元素个数
int getRange(int u, int v, int l, int r)
{
    return getRange(root[u - 1], root[v], Start, Last, l, r);
}

// 查询区间  $[u, v]$  中第  $k$  小的元素
int getKth(int u, int v, int k)
{
    return getKth(root[u - 1], root[v], Start, Last, k);
}
T getKthSum(int u, int v, int k)
{
    return getKthSum(root[u - 1], root[v], Start, Last, k);
}
};

```

0.7.2 维护历史版本

在维护历史版本的主席树中，我们可以支持在新开版本时实现区间加，使用标记永久化的技巧即可。

```

template <class T>
class PresidentTree
{
public:
    // 构造函数：使用给定数组  $a$  初始化主席树， $capacity$  指定预分配空间大小
    PresidentTree(const std::vector<int> &a, int capacity)
        : n(a.size()), // 数组长度
          root(n), // 各版本根节点数组
          lc(capacity + 1), // 左子节点数组 (+1 防止越界)
          rc(capacity + 1), // 右子节点数组
          sum(capacity + 1) // 节点和数组
    {
        // 递归构建初始版本线段树的  $\lambda$  函数
        std::function<void(int &, int, int)> build = [&](int &u, int l, int r)
        {
            u = ++idx; // 分配新节点， $idx$  从 1 开始
            if (l == r) // 叶子节点
            {
                sum[u] = a[l]; // 存储数组值
                return;
            }
            int mid = l + r >> 1; // 计算中点
            build(lc[u], l, mid); // 递归构建左子树
            build(rc[u], mid + 1, r); // 递归构建右子树
            pushup(u); // 合并子节点信息
        }
    }
};

```

```

    };
    build(root[0], 1, n - 1); // 构建初始版本 (版本 0), 基于数组索引
}

private:
    int idx = 0;           // 节点计数器, 从 1 开始分配
    int n;                 // 数组长度
    std::vector<int> root; // 各版本根节点数组
    std::vector<int> lc;   // 左子节点数组
    std::vector<int> rc;   // 右子节点数组
    std::vector<T> sum;    // 节点和数组 (使用 long long 类型)

    // 向上合并子节点信息
    void pushup(int u)
    {
        sum[u] = sum[lc[u]] + sum[rc[u]]; // 当前节点和为左右子节点和之和
    }

    // 插入新版本: 基于版本 v 创建新版本 u, 在 pos 位置更新值为 x
    void insertNewVersion(int &u, int v, int l, int r, int pos, int x)
    {
        u = ++idx; // 分配新节点 (关键: 必须创建新节点以实现可持久化)

        // 复制旧版本 v 的信息
        lc[u] = lc[v];
        rc[u] = rc[v];
        sum[u] = sum[v];

        if (l == r) // 到达目标位置
        {
            sum[u] = x; // 更新值
            return;
        }

        int mid = l + r >> 1; // 计算中点
        if (pos <= mid) // 目标在左子树
        {
            // 递归处理左子树, 创建新路径
            insertNewVersion(lc[u], lc[v], l, mid, pos, x);
        }
        else // 目标在右子树
        {
            // 递归处理右子树, 创建新路径
            insertNewVersion(rc[u], rc[v], mid + 1, r, pos, x);
        }
        pushup(u); // 更新当前节点信息
    }
};

```

0.7.3 维护树上信息

我们在线段树合并部分其实已经涉及此问题，我们认为线段树合并就是一种树上主席树类似的结构。

但是线段树合并长于维护子树信息，在维护链上问题时几乎束手无策，因此我们需要想新的办法

我们引出树上主席树，其本质仍然是主席树，只不过把区间变成了树，树实际上可以转化为不同的序列，我们按特定的顺序建主席树的方法，称为树上主席树，分两种，分别为 *DFS* 序和 *BFS* 序

0.7.3.1 DFS 序 *DFS* 序建立的主席树可以解决树链上的值域问题，我们按 *dfn* 序遍历的顺序来建树，每次基于的版本来自其父节点，那么怎么转化为树链呢？我们采用树上差分的思想即可。

0.7.3.2 BFS 序 按 *dep* 排序，也就是我们以 *dep* 为依据加点，每个节点基于的版本是上一个 *dep* 相同的节点，如果没有则基于 *dep* - 1 新开版本即可。

主要面向的问题是，对节点 *u*，询问其子树内深度小于等于 *d* 的信息，注意到虽然完整子树的 *dfn* 序连续，但是只截取部分深度是不连续的。但按我们这样建立的主席树，相同深度的节点将是同一个版本，也就可以解决这一类问题。但是其他子树内也存在深度小于等于 *d* 的节点，怎么办呢？注意到主席树实际上可以限制两维信息，分别是版本区间和内层值域区间，我们只需要限制值域为 $[dfn[u], dfn[u] + siz[u] - 1]$ 即可

0.8 字典树

0.8.1 字典树

在 insert 和 query 中初始 p 为 0, 节点 p 不存信息, 其 26(小写 26 个英文字母) 个 son 指向所有字符串的开头, 根据需要维护的信息修改 node 的变量 和 insert 和 query

```
struct Trie {
    struct node {
        std::array<int, 26> son;
        int cnt = 0;
        node() : son{} {}
    };

    std::vector<node> t;

    Trie() {
        init();
    }

    void init() {
        t.assign(1, node());
    }

    int newnode() {
        t.emplace_back();
        return t.size() - 1;
    }

    void insert(const std::string &val) {
        int p = 0;
        for (char c: val) {
            int ne = c - 'a';
            if (t[p].son[ne] == 0) {
                t[p].son[ne] = newnode();
            }
            p = t[p].son[ne];
        }
        t[p].cnt++;
    }

    int query(const std::string &val) {
        int p = 0;
        for (char c: val) {
            int ne = c - 'a';
            if (t[p].son[ne] == 0) {
                return 0;
            }
            p = t[p].son[ne];
        }
        return t[p].cnt;
    }
};
```


0.8.2 可持久化字典树

经典应用: 给定一个数 x , 在区间 $[l, r]$ 中找一个数 y 使得 $x \oplus y$ 的值最大, 下面的模板以 01Trie 举例, t 数组存树的信息, $ver[i]$ 表示第 i 个版本根节点的编号

```
template<class T>
struct PerTrie {
    struct node{
        array<int, 2> son;
        node():son{{}}
    };
    int tot = 0;
    std::vector<node> t;
    std::vector<int> ver, cnt;
    T inf = std::numeric_limits<T>::max(); // 可根据数据范围自行改
    PerTrie() {
        init();
    }
    void init() {
        t.assign(1, {});
        ver.assign(1, {});
        cnt.assign(1, {});
    }
    // y 是上一个版本的根节点编号, 不要传错参数
    void add(T x, T y = 0) {
        int p = ++tot;
        ver.emplace_back(p);
        for (int i = std::__lg(inf); i >= 0; i--) {
            int q = x >> i & 1; // q 表示下一个二进制位
            while (p >= t.size()) t.emplace_back(); // 因为可能给某个节点已经分配了编号但是还没有分配空间
            t[p].son[q^1] = t[y].son[q^1];
            t[p].son[q] = ++tot;
            p = t[p].son[q], y = t[y].son[q];
            while (p >= cnt.size()) cnt.emplace_back();
            cnt[p] = cnt[y] + 1;
        }
    }
    // 求 x 与区间 [l+1, r] 中某个元素 xor 的最大值
    // l, r 注意要传入对应的根节点, 即 tr.version[l], tr.version[r]
    T query(int l, int r, T x) {
        T ans = 0;
        for (int i = std::__lg(inf); i >= 0; i--) {
            int q = x >> i & 1;
            // 没有被分配节点的编号为 0, cnt[0] = 0
            if (cnt[t[r].son[q^1]] > cnt[t[l].son[q^1]]) {
                r = t[r].son[q^1], l = t[l].son[q^1], ans += 1ll << i;
            } else {
                r = t[r].son[q], l = t[l].son[q];
            }
        }
        return ans;
    }
    int version(int p) {
        return ver[p];
    }
    int size() { return ver.size(); }
};
```

例题

给定一个非负整数序列 $\{a\}$ ，初始长度为 N

有 M 个操作，有以下两种操作类型：

1. A x : 添加操作，表示在序列末尾添加一个数 x ，序列的长度 N 加 1。
2. Q $l\ r\ x$: 询问操作，你需要找到一个位置 p ，满足 $l \leq p \leq r$ ，使得： $a[p] \oplus a[p+1] \oplus \dots \oplus a[N] \oplus x$ 最大，输出最大值。

```
void solve(){
    int n,m;cin>>n>>m;
    PerTrie<int> tr;
    tr.add(0,0); // 此时根节点 1 已经被这个占用了，所有后面的都要偏移一位
    vector<int> a(n+1);
    for(int i = 1;i<=n;i++){
        cin>>a[i];a[i] ^= a[i-1];
        tr.add(a[i],tr.version(i));
    }
    while(m--){
        char o;cin>>o;
        if(o == 'A'){
            int x;cin>>x;
            a.emplace_back(x^a.back());
            n++;
            tr.add(a[n],tr.version(n));
        }else{
            int l,r,x;cin>>l>>r>>x;
            cout<<tr.query(tr.version(l-1), tr.version(r), a[n]^x)<<"\n";
        }
    }
}
```

0.9 并查集

0.9.1 普通并查集

只维护了 `fa` 以及每个集合的元素数量，维护其他信息可以手搓

```
struct DSU {
    std::vector<int> fa, siz;

    DSU() {}

    DSU(int n) {
        init(n);
    }

    void init(int n) {
        fa.resize(n);
        std::iota(fa.begin(), fa.end(), 0);
        siz.assign(n, 1);
    }

    int find(int x) {
        while (x != fa[x]) {
            x = fa[x] = fa[fa[x]];
        }
        return x;
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }
    // 把 y 所在集合的 fa 都指向 x 所在集合的 fa
    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) {
            return false;
        }
        siz[x] += siz[y];
        fa[y] = x;
        return true;
    }

    int size(int x) {
        return siz[find(x)];
    }
};
```

0.9.2 带权并查集

```
template<class T>
struct DSU {
    std::vector<int> fa, siz;
    std::vector<T> dis;
    DSU() {}
    DSU(int n) {
        init(n);
    }
    void init(int n) {
        fa.resize(n);
        dis.assign(n, 0);
        std::iota(fa.begin(), fa.end(), 0);
        siz.assign(n, 1);
    }

    int find(int x) {
        if (fa[x] == x) return x;
        int tmp = find(fa[x]);
        dis[x] += dis[fa[x]];
        return fa[x] = tmp;
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }
    // 把所有的 y 集合的元素接到 x 后面并维护到达 x 集合根节点的距离
    // 如果用到加法一定要注意负数取模 !!!
    bool merge(int x, int y, int v) {
        int fx = find(x);
        int fy = find(y);
        if (fx == fy) {
            return false;
        }
        // 维护先对位置一般  $d[fy] = d[x] + v - d[y]$ ;
        dis[fy] = siz[fx];
        siz[fx] += siz[fy];
        fa[fy] = fx;
        return true;
    }

    int size(int x) {
        return siz[find(x)];
    }

    T dist(int x, int y) {
        if (find(x) != find(y)) return -1;
        return dis[y] - dis[x];
    }

    T dist(int x) {
        return find(x), dis[x];
    }
};
```

例题:

Alice 和 Bob 在玩一个游戏：他写一个由 0 和 1 组成的序列。Alice 选其中的一段（比如第 3 位到第 5 位），问他这段里面有奇数个 1 还是偶数个 1。Bob 回答你的问题，然后 Alice 继续问。Alice 要检查 Bob 的答案，指出在 Bob 的第几个回答一定有问题。有问题的意思就是存在一个 01 序列满足这个回答前的所有回答，而且不存在序列满足这个回答前的所有回答及这个回答。

解：只需要维护 $d[i]$ 表示二进制字符串中第 i 个位置前 1 的个数，给定 l, r ，就 $\text{merge}(l-1, r)$ ，每次检查 $d[r]-d[l-1] \bmod 2$ 是不是 1 或者 0 就可以，此时 merge 操作中用 $d[\text{fy}] = d[\text{x}] + v - d[\text{y}]$ 维护相对距离

0.9.3 可撤销并查集

```
struct DSU {
    int n;
    std::vector<int> siz, f;
    std::vector<std::array<int, 2>> history;
    DSU(int _n = 1) {
        init(_n);
    }
    void init(int _n) {
        this->n = _n;
        siz.assign(n, 1);
        f.resize(n);
        std::iota(f.begin(), f.end(), 0);
    }
    int find(int x) {
        while (f[x] != x) {
            x = f[x];
        }
        return x;
    }
    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) {
            return false;
        }
        if (siz[x] < siz[y]) {
            std::swap(x, y);
        }
        history.push_back({x, y});
        siz[x] += siz[y];
        f[y] = x;
        return true;
    }
    int time() {
        return history.size();
    }
    void rollback(int tm) {
        while (history.size() > tm) {
            auto [x, y] = history.back();
            history.pop_back();
            f[y] = y;
            siz[x] -= siz[y];
        }
    }
    int size(int p) {
        return siz[find(p)];
    }
    bool same(int u, int v) {
        return find(u) == find(v);
    }
};
```

0.10 线性基

0.10.1 普通线性基

线性基是通过一系列的线性变换/异或变换把一系列的向量转化为一组线性无关的基 Base

在整数的异或线性基中，向量即为每个数的二进制表示，每个二进制位对应向量的一个位

假如说原本的数组通过异或得到的所有数字的集合为 S ，通过导出的 Base 我们可以贪心解决异或最大值， k 大值等问题。

当使用的 G 为 bitset 时稍微改改就行，比如：

$(!Base[i]) \rightarrow (Base[i] == 0)$ (此时 0 会转换成 $bitset<>(0)$)

$(x \& (1ll \ll i)) \rightarrow ((x >> i)[0])$

其中查询第 k 小复杂度是 $\log(\inf) * \log(\inf)$ ，其他操作都是 $\log(\inf)$

```
/**
 * 线性基模板
 * T: 线性基的位数
 * G: 元素类型，通常为 int 或 long long
 */
template <int T, class G> struct LineBase {
    std::vector<G> Base; // 存储线性基的数组
    bool flag = false; // 标记是否存在无法插入的元素 (即是否存在异或和为 0 的子集)
    // 从数组初始化线性基
    LineBase(const std::vector<G>& a) : Base(T, 0) {
        for (int i = 1; i < a.size(); ++i) { insert(a[i]); }
    }
    // 默认构造函数
    LineBase() : Base(T, 0) {}
    // 插入元素 x 到线性基中
    void insert(G x) {
        for (int i = T - 1; ~i; i--) {
            if (x & (1ll << i)) // 如果 x 的第 i 位为 1
            {
                if (!Base[i]) // 如果线性基的第 i 位为空
                {
                    Base[i] = x; // 插入 x
                    return;
                } else {
                    x ^= Base[i]; // 否则消去 x 的第 i 位
                }
            }
        }
        flag = true; // 执行到这里说明 x 被消为 0，存在异或和为 0 的子集
    }
    // 判断 x 能否由线性基中的元素异或得到
    bool check(G x) {
        for (int i = T - 1; ~i; --i) {
            if (x & (1ll << i)) // 如果 x 的第 i 位为 1
            {
                if (!Base[i]) // 线性基的第 i 位为空，无法表示
                {
                    return false;
                } else {
                    x ^= Base[i]; // 消元操作
                }
            }
        }
    }
}
```

```

        return true; // 能消为 0, 说明可以表示
    }

    // 查询线性基能表示的最大异或和
    G qmax() {
        G res = 0;
        for (int i = T - 1; ~i; --i) {
            res = std::max(res, res ^ Base[i]); // 贪心选择能使结果更大的元素
        }
        return res;
    }

    // 查询线性基能表示的最小异或和
    G qmin() {
        if (flag) // 存在异或和为 0 的子集
        {
            return 0;
        }
        for (int i = 0; i < T; ++i) {
            if (Base[i]) // 找到第一个非零元素即为最小值
            {
                return Base[i];
            }
        }
        return 0; // 线性基为空的情况
    }

    // 查询第 k 小的异或和, 复杂度  $T^2$ 
    G val(G k) {
        std::vector<G> tmp(T);
        G res = 0;
        G cnt = 0;
        k -= flag; // 减去 0 的情况
        if (!k) { return 0; }

        // 重构线性基, 使其变为上三角形式
        for (int i = 0; i < T; ++i) {
            G now = Base[i];
            for (int j = i - 1; ~j; j--) {
                if (now & (1ll << j)) { now ^= Base[j]; }
            }
            if (now) { tmp[cnt++] = now; }
        }

        if (k >= (1ll << cnt)) // 超出范围
        {
            return -1;
        }

        // 根据 k 的二进制位选择对应的基向量
        for (int i = 0; i < cnt; ++i) {
            if (k & (1ll << i)) { res ^= tmp[i]; }
        }
        return res;
    }
}

```



```

// 查询 val 的排名 (即比 val 小的异或和数量 +1) , 复杂度 T
G rank(int val) {
    if(!check(val)){
        return -1;
    }
    if (val > qmax()) // 超出最大值
    {
        return -1;
    }

    G res = 0;
    if (flag) // 存在 0 的情况
    {
        res++;
    }

    std::vector<G> st(T);
    int cnt = 0;

    // 收集所有非零基向量的位置
    for (int i = 0; i < T; ++i) {
        if (Base[i]) { st[cnt++] = i; }
    }

    // 计算排名
    for (int i = 0; i < cnt; ++i) {
        if (val >> st[i] & 1) {
            res += 1ll << i; // 累加贡献
        }
    }
    return res;
}

// 查询线性基能表示的不同异或和的数量
G num() {
    return rank(qmax()); // 即最大值的排名
}
};

```

0.10.2 前缀线性基

可以理解为可持久化线性基，本质就是建立了不同版本的线性基。我们用 $p[idx][i]$ 表示第 idx 个版本的线性基第 i 个位的基，用 $pos[idx][i]$ 表示该基最晚出现的位置（贪心），如果新加入的数可以导出某个基且出现位置更晚，我们就和原本的基贪心的交换位置。因为我们对于区间询问 $[l, r]$ ，我们会查 r 版本的线性基，如果某个位置 i 的基的最晚出现位置 $\geq l$ ，就说明这个基是可用的。

注意构造函数传入的参数数组是 1-base 的

```
/**
 * 可持久化线性基模板
 * Z: 线性基的位数
 * G: 元素类型，通常为 int 或 long long
 */
template <int Z, class G>
struct PreLineBase
{
    // p[i][j]: 第 i 个版本中第 j 位的线性基元素
    // pos[i][j]: 第 i 个版本中第 j 位线性基元素的插入位置
    std::vector<std::array<int, Z>> p;
    std::vector<std::array<int, Z>> pos;
    // 构造函数，从数组 a 初始化可持久化线性基
    // 数组 a 为 1-base
    PreLineBase(const std::vector<G> &a) : p(a.size()), pos(a.size())
    {
        // 初始化第 0 个版本
        for (int i = 0; i < Z; ++i){
            p[0][i] = pos[0][i] = 0;
        }
        // 逐个插入元素，生成新版本
        for (int i = 1; i < a.size(); ++i)
        {
            insert(a[i], i);
        }
    }

    // 插入元素 x, 生成新版本 idx
    void insert(G x, int idx)
    {
        // 继承前一个版本的数据
        p[idx] = p[idx - 1];
        pos[idx] = pos[idx - 1];
        int cur = idx; // 当前元素的插入位置
        // 从高位到低位处理每一位
        for (int i = Z - 1; i >= 0; --i)
        {
            if (x >> i & 1) // 如果 x 的第 i 位为 1
            {
                if (!p[idx][i]) // 如果线性基的第 i 位为空
                {
                    p[idx][i] = x; // 插入 x
                    pos[idx][i] = cur; // 记录插入位置
                    break;
                }
                // 贪心策略：保留插入位置较后的元素
                if (pos[idx][i] < cur)
                {
                    std::swap(p[idx][i], x);
                    std::swap(pos[idx][i], cur);
                }
            }
        }
    }
};
```

```

        }
        // 消元操作
        x ^= p[idx][i];
    }
}

// 查询区间 [l, r] 内元素能组成的最大异或和
G querymax(int l, int r)
{
    G ans = 0;
    // 从高位到低位贪心选择
    for (int i = Z - 1; i >= 0; --i)
    {
        // 只考虑插入位置在 [l, r] 内的元素
        if (pos[r][i] >= l)
        {
            ans = std::max(ans, ans ^ p[r][i]);
        }
    }
    return ans;
}

// 判断区间 [l, r] 内元素能否异或得到 x
bool isexist(int l, int r, G x)
{
    // 从高位到低位处理
    for (int i = Z - 1; i >= 0; --i)
    {
        if ((x >> i & 1) and pos[r][i] < l)
        {
            // 如果 x 的第 i 位为 1, 但对应线性基元素的插入位置小于 l, 无法生成
            return false;
        }
        else if (x >> i & 1)
        {
            // 消元操作
            x ^= p[r][i];
        }
    }
    // 如果能消为 0, 则可以生成
    return x == 0;
}

// 判断区间 [l, r] 内元素能否异或得到 0
bool isexist0(int l, int r)
{
    int cnt = 0;
    // 从高位到低位处理
    for (int i = Z - 1; i >= 0; --i){
        if (pos[r][i] >= l){
            cnt++;
        }
    }
    // 有一个元素没有贡献就说明被消掉了
    return cnt < r - l + 1;
}
};

```

0.11 ST 表

0.11.0.1 返回权值 (返回区间最大值, 需要最小值就改 max)

```
template <class T>
class ST
{
    std::vector<std::vector<T>> st;

public:
    ST(const std::vector<T> &a)
    {
        int n = a.size() - 1, logn = std::__lg(n);
        st.assign(logn + 1, std::vector<int>(n + 1));
        for (int i = 1; i <= n; ++i)
        {
            st[0][i] = a[i];
        }
        for (int j = 1; j <= logn; ++j)
        {
            for (int i = 1; i + (1 << j) - 1 <= n; ++i) // n-i+1=(1<<j) 是边界
            {
                st[j][i] = std::max(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
            } // i 是当前位置 j 是向右扩展 (1<<j) 个数字
        }
    }

    T operator()(int l, int r)
    {
        int log = std::__lg(r - l + 1);
        return std::max(st[log][l], st[log][r - (1 << log) + 1]);
    }
};
```

0.11.0.2 返回位置 (返回权值最大的下标)

```
template <class T>
class ST
{
    std::vector<std::vector<int>>> st;
    std::vector<T> A;

public:
    ST(const std::vector<T> &a)
    {
        A = a;
        int n = a.size() - 1, logn = std::__lg(n);
        st.assign(logn + 1, std::vector<int>(n + 1));
        for (int i = 1; i <= n; ++i)
        {
            st[0][i] = i;
        }
        for (int j = 1; j <= logn; ++j)
        {
            for (int i = 1; i + (1 << j) - 1 <= n; ++i)
            {
                st[j][i] = (a[st[j - 1][i]] > a[st[j - 1][i + (1 << (j - 1))]])
                    ? st[j - 1][i] : st[j - 1][i + (1 << (j - 1))]);
            }
        }
    }

    int operator()(int l, int r)
    {
        int log = std::__lg(r - l + 1);
        return (A[st[log][l]] > A[st[log][r - (1 << log) + 1]])
            ? st[log][l] : st[log][r - (1 << log) + 1];
    }
};
```

0.12 根号算法

0.12.1 莫队

莫队是针对形如 Q 次询问，每次给定 L, R 询问区间信息的暴力算法，其思想一定程度上基于分块，通过把序列分块的方式，我们离线所有询问，询问区间 $[L, R]$ 按左端点所在块编号为第一关键字，右端点为第二关键字排序，然后操控指针移动暴力求解。可以证明在这样的操作顺序下，指针移动次数是 $\mathcal{O}(n \sqrt{n})$ ， $\mathcal{O}(n \sqrt{n})$

0.12.1.1 普通莫队

```
struct Query
{
    int l, r, id;
};
template <class T>
class RollbackMoTeam
{
public:
    RollbackMoTeam(std::vector<T> &a) : n(a.size() - 1), B(sqrt(2 * n)), val(a)
    {
        blockNum = n / B + bool(n % B);
        res = last = 0;
    }

    int be(int x)
    {
        return (x - 1) / B + 1;
    }
    void addQuery(int l, int r, int id)
    {
        Q.push_back(Query{1, r, id});
    }
    void add(T x) // add 和 del 只要一个
    {
    }
    // void del(T x)
    // {
    // }

    void clear()
    {
        res = last = 0;
        // 这里要清空之前的所有信息
    }
    T calc(int l, int r)
    {
    }

    std::vector<T> work()
    {
        std::sort(begin(Q), end(Q), [&](const auto &s, const auto &t)
            { return be(s.l) == be(t.l) ? s.r < t.r : s.l < t.l; });

        int idx = 0;
        for (int i = 1; i <= blockNum; ++i) // 对每个块单独考虑
        {
            clear();
            int R = std::min(B * i, n);
```

```

int pl = R + 1, pr = R;
for (; be(Q[idx].l) == i; ++idx)
{
    auto [l, r, id] = Q[idx];
    if (r - l + 1 <= B)
    {
        ans[id] = calc(l, r);
    }
    else
    {
        while (pr < r)
        {
            add(val[++pr]); // 右扩展
        }
        last = res; // 结果存为 last
        while (pl > l)
        {
            add(val[--pl]); // 左扩展
        }
        ans[id] = res; // 结果存入答案
        while (l <= R)
        {
            // 清空信息 add 加了什么就清空什么
            l++;
        }
        res = last; // 回滚结果
    }
}
}
return ans;
}

private:
int n, B, blockNum;
std::vector<T> val;
std::vector<T> ans;
std::vector<Query> Q;
T res, last;
};

```

0.12.1.2 回滚莫队

- **右扩展 (pr)**: 是永久的, 直到当前块结束。
- **左扩展 (pl)**: 是临时的, 处理完一个询问就要撤回。

```
struct Query
{
    int l, r, id;
};
template <class T>
class RollbackMoTeam
{
public:
    RollbackMoTeam(std::vector<T> &a) : n(a.size() - 1), B(sqrt(2 * n)), val(a)
    {
        blockNum = n / B + bool(n % B);
        res = last = 0;
        ans.push_back(0);
    }

    int be(int x)
    {
        return (x - 1) / B + 1;
    }
    void addQuery(int l, int r, int id) // 记得给 ans 分配空间
    {
        Q.push_back(Query{l, r, id});
        ans.push_back(0);
    }
    void add(T x) // add 和 del 只要一个, 加右边和加左边的操作可能不同, 可以分开写
    {
    }
    // void del(T x)
    // {
    // }

    void clear()
    {
        res = last = 0;
        // 这里要清空之前的所有信息
    }
    T calc(int l, int r) // 暴力处理长度小于等于 B 的区间
    {
    }

    std::vector<T> work()
    {
        std::sort(begin(Q), end(Q), [&](const auto &s, const auto &t)
            { return be(s.l) == be(t.l) ? s.r < t.r : s.l < t.l; });

        int idx = 0;
        for (int i = 1; i <= blockNum; ++i) // 对每个块单独考虑
        {
            clear();
            int R = std::min(B * i, n);
            int pl = R + 1, pr = R;
```



```

for (;idx<size(Q) and be(Q[idx].l) == i; ++idx)
{
    auto [l, r, id] = Q[idx];
    if (r - l + 1 <= B)
    {
        ans[id] = calc(l, r);
    }
    else
    {
        while (pr < r)
        {
            add(val[++pr]); // 右扩展
        }
        last = res; // 结果存为 last
        while (pl > l)
        {
            add(val[--pl]); // 左扩展
        }
        ans[id] = res; // 结果存入答案
        while (pl <= R)
        {
            // 清空信息 add 加了什么就清空什么
            pl++;
        }
        res = last; // 回滚结果
    }
}
}
return ans;
}

private:
    int n, B, blockNum;
    std::vector<T> val;
    std::vector<T> ans;
    std::vector<Query> Q;
    T res, last;
};

```

0.13 堆

对于 `std::priority_queue` 不再过多介绍

0.13.1 可删堆

(功能完全可以由 `multiset` 取代, 但是 `set` 的常数很大, 如果被卡可以这样优化一下)

用两个 `std::priority_queue` 来实现, 其中一个用来存要删除的数字, 每次取 `top()` 时判断是否需要弹出即可。

```
template<class T>
struct Heap
{
    Heap() {}
    std::priority_queue<T> h, del;
    void erase(T x)
    {
        del.push(x);
    }
    void add(T x)
    {
        h.push(x);
    }
    int size()
    {
        return h.size() - del.size();
    }
    void pop()
    {
        assert(size());
        while (del.size() and del.top() == h.top())
        {
            h.pop();
            del.pop();
        }
        h.pop();
    }
    T top()
    {
        assert(size());
        while (del.size() and del.top() == h.top())
        {
            h.pop();
            del.pop();
        }
        return h.top();
    }
};
```

0.13.2 对顶堆

动态维护序列中第 k 大的数字，使用一个大根堆和一个小根堆动态维护即可。

堆实现 只支持加数字

```
template <class T>
struct Heap
{
    std::priority_queue<T> a;
    std::priority_queue<T, std::vector<T>, std::greater<T>> b;

    void add(T x)
    {
        if (b.empty() or x >= b.top())
        {
            b.push(x);
        }
        else
        {
            a.push(x);
        }
    }
    T query(int k)
    {
        while (b.size() > k)
        {
            a.push(b.top());
            b.pop();
        }
        while (b.size() < k)
        {
            b.push(a.top());
            a.pop();
        }
        return b.top();
    }
};
```

multiset 实现，额外支持删数字

```
template <class T>
struct Heap
{
    std::multiset<T> a;
    std::multiset<T> b;
    void add(T x)
    {
        if (b.empty() || x >= *b.begin()){
            b.insert(x);
        }
        else{
            a.insert(x);
        }
    }
    // 删除元素 (如果存在)
    bool remove(T x)
    {
        // 先检查是否在 b 中
        auto it = b.find(x);
        if (it != b.end()){
            b.erase(it);
            return true;
        }
        // 再检查是否在 a 中
        it = a.find(x);
        if (it != a.end()){
            a.erase(it);
            return true;
        }
        // 元素不存在
        return false;
    }
    T query(int k)
    {
        while (b.size() > k){
            auto val = *b.begin();
            b.erase(b.begin());
            a.insert(val);
        }
        while (b.size() < k){
            auto val = *a.rbegin();
            a.erase(std::prev(a.end()));
            b.insert(val);
        }
        return *b.begin();
    }
    int size() const
    {
        return a.size() + b.size();
    }
};
```

0.13.3 可并堆 (pbds 实现)

请注意大根堆是 `std::less`，小根堆是 `std::greater` 洛谷 p3377

使用教程：

我们在 `solve` 函数内自己开一个迭代器数组，当加上某个数，删除某个数，修改某个数，需要根据返回的 `it` 自动更改迭代器为新迭代器或者 `nullptr`，如果是删除堆顶，我们一开始传入数据时传入每个数字的下标，删除堆顶时手动把堆顶的下标对应的迭代器置为 `nullptr`，这样可以任意操作堆中的任一元素，而不是只能访问堆顶

操作	二项堆 (Binomial)	配对堆 (Pairing)	备注
push	$O(\log n)$ (最坏)	$O(1)$	配对堆直接挂载，二项堆可能要多次“进位”
join	$O(\log n)$ (最坏)	$O(1)$	配对堆的神技，也是你模板里最核心的区别
top	$O(\log n)$ 或 $O(1)$	$O(1)$	二项堆需要扫描森林中所有树的根（除非维护额外指针）
pop	$O(\log n)$	$O(\log n)$ (均摊)	性能瓶颈都在这里
modify	$O(\log n)$	$O(\log n)$ (均摊)	配对堆实际常数极小

```
#include <bits/stdc++.h>
#include <bits/extc++.h> // 包含 pbds 库 (部分编译器可能需要用 #include <ext/pb_ds/priority_queue.hpp>)
namespace pbds = __gnu_pbds;
template <typename T, typename Compare = std::less<T>>
class mergeHeap {
private:
    // 也可以用 pbds::binomial_heap_tag, 具体区别看表格
    using Heap = pbds::priority_queue<T, Compare, pbds::pairing_heap_tag>;
    Heap heap_;
public:
    using it = typename Heap::point_iterator;
    mergeHeap() = default;
    // 返回插入元素位置的迭代器
    it push(const T &value) { return heap_.push(value); }
    const T &top() const { return heap_.top(); }
    void pop() { heap_.pop(); }
    void erase(it t) { heap_.erase(t); }
    void merge(mergeHeap &other) { // 把 other 堆合并到自身, other 变成空堆
        heap_.join(other.heap_); // pb_ds 内部会处理 other 是否为空
    }
    bool empty() const { return heap_.empty(); }
    size_t size() const { return heap_.size(); }
    void clear() { heap_.clear(); }
    // 使用内置的 modify 性能更好
    // modify 操作并不会更改这个元素位置的迭代器, 只是改变指向关系
    it modify(it t, const T &new_value) {
        heap_.modify(t, new_value);
        return t;
    }
};

template <typename T>
using MinBinomialHeap = mergeHeap<T, std::greater<T>>;
template <typename T>
using MaxBinomialHeap = mergeHeap<T, std::less<T>>;
```

0.14 其他

0.14.1 珂朵莉树

珂朵莉树 (ODT) 用于维护序列, 其核心是一个 $Node(l, r, val)$ 的结构体 *set* 容器, 其中 *val* 是 *mutable* 类型使得在 *set* 中该值可以被修改。那么 *set* 会自动按照左端点升序排序, 每个节点都代表一个区间, 该区间内每个位置的值相同, 都为对应的 *val*。

其核心操作为 *split* 和 *assign*, 分裂和覆盖, 在数据随机的情况下 *assign* 可以保证区间数量不会很多进而保证复杂度。

```
template <class T>
class ODT
{
private:
    const T inf = std::numeric_limits<T>::max() / 2;
    struct Node
    {
        int l, r;          // 区间左右端点 (闭区间  $[l, r]$ )
        mutable T val;     // 值 (mutable 允许在 set 中修改值)

        // 用于 set 排序的运算符: 按左端点 l 从小到大排序
        bool operator<(const Node &a) const
        {
            return l < a.l;
        }

        // 构造函数: 默认构造一个单点区间或指定区间
        Node(int L, int R, T Val) : l(L), r(R), val(Val) {}
    };

    std::set<Node> s;          // 有序集合存储所有区间节点, 自动按左端点排序
    using IT = typename std::set<Node>::iterator; // 使用 using 代替 #define

public:
    ODT()
    {
        // 初始化时插入一个虚拟节点作为哨兵 (防止越界)
        // 哨兵节点的右边界设为极大值, 确保所有合法区间都在其左侧
        s.insert(Node(0, inf, 0)); // 假设数据范围为  $0 \sim 1e9$ , 可根据实际需求调整
    }

    IT split(int pos)
    {
        /*
         * 功能: 将区间在 pos 处分裂为  $[l, pos-1]$  和  $[pos, r]$  两段
         * 返回值: 后半段  $[pos, r]$  的迭代器
         * 原理: 利用 set 的有序性找到第一个左端点  $\geq pos$  的节点,
         *       若该节点左端点等于 pos, 说明已存在分裂点, 直接返回;
         *       否则分裂前一个节点, 生成两个新区间
         */
        IT it = s.lower_bound(Node(pos)); // 找到第一个左端点  $\geq pos$  的节点
        if (it != s.end() && it->l == pos)
        {
            // 若存在左端点为 pos 的节点, 直接返回 (已分裂过)
            return it;
        }
        // 否则需要分裂前一个节点
        it--;
        // 移动到前一个节点 (该节点的  $l < pos \leq r$ )
        int L = it->l, R = it->r; // 原节点的左右边界
```

```

    T Val = it->val;           // 原节点的值
    s.erase(it);             // 删除原节点
    // 插入左半段 [L, pos-1]
    s.insert(Node(L, pos - 1, Val));
    // 插入右半段 [pos, R] 并返回其迭代器
    return s.insert(Node(pos, R, Val)).first;
}

void assign(int l, int r, T val)
{
    /*
     * 功能: 将区间 [l, r] 内的所有值覆盖为 val
     * 步骤:
     * 1. 分裂右端点 r+1, 得到后半段迭代器 itr
     * 2. 分裂左端点 l, 得到前半段迭代器 itl
     * 3. 删除 [itl, itr) 区间内的所有节点 (即原 [l, r] 区间的所有分段)
     * 4. 插入新节点 [l, r, val] 表示覆盖后的区间
     */
    IT itr = split(r + 1);     // 分裂出 [r+1, ...]
    IT itl = split(l);         // 分裂出 [l, ...]
    s.erase(itl, itr);        // 删除 [l, r] 区间内的所有旧分段
    s.insert(Node(l, r, val)); // 插入新的覆盖区间
}

void add(int l, int r, T val)
{
    /*
     * 功能: 将区间 [l, r] 内的所有值增加 val
     * 步骤:
     * 1. 分裂区间两端点, 确保 [l, r] 是若干连续节点的集合
     * 2. 遍历 [itl, itr) 区间内的所有节点, 逐个增加 val
     */
    IT itr = split(r + 1); // 分裂右端点
    IT itl = split(l);     // 分裂左端点
    for (; itl != itr; ++itl)
    {
        itl->val += val; // 对每个节点的值进行加法操作
    }
}

T kth(int l, int r, int k)
{
    /*
     * 功能: 查询区间 [l, r] 内的第 k 小值 (k 从 1 开始)
     * 步骤:
     * 1. 分裂区间两端点, 获取所有包含在 [l, r] 内的节点
     * 2. 收集这些节点的值和长度, 按值排序
     * 3. 遍历排序后的节点, 累加长度直到找到第 k 小值
     */
    std::vector<std::pair<T, int>> vp; // 存储 (值, 区间长度)
    IT itr = split(r + 1);           // 分裂右端点
    IT itl = split(l);               // 分裂左端点
    for (; itl != itr; ++itl)
    {
        vp.push_back({itl->val, itl->r - itl->l + 1});
    }
}

```

```

std::sort(vp.begin(), vp.end()); // 按值从小到大排序
for (auto [val, len] : vp)
{
    k -= len; // 扣除当前区间的长度
    if (k <= 0)
    { // 找到第 k 小值
        return val;
    }
}
return -1; // 若未找到 (理论上不应发生)
}

T RangeQuery(int l, int r)
{
    /*
    * 功能: 查询区间 [l, r] 内所有值的和
    * 步骤:
    * 1. 分裂区间两 endpoints, 获取所有包含在 [l, r] 内的节点
    * 2. 遍历节点, 计算每个节点的区间长度 * 值, 累加求和
    */
    IT itr = split(r + 1); // 分裂右端点
    IT itl = split(l);     // 分裂左端点
    T res = 0;
    for (; itl != itr; ++itl)
    {
        res += (itl->r - itl->l + 1) * itl->val; // 区间长度 * 值
    }
    return res;
}
};

```


0.14.2 笛卡尔树

笛卡尔树是把一个序列建立成一颗二叉搜索树，整棵树的中序遍历为长度为 n 的顺序排列（即 $(1, 2, 3, 4 \dots n)$ ），同时满足堆的性质，以小根堆为例，也就是 $val[x] \leq val[lc[x]]$, $val[x] \leq val[rc[x]]$

性质：笛卡尔树每个点的子树下标连续，同时子树的根是子树中值最小/最大的节点。

构建过程（小根堆笛卡尔树）：我们按元素的下标插入，设当前插入的下标为 i ，显然新插入的元素应该在树的最右边，那么我们用一个栈维护最右边的树链，通过不断弹出栈顶来从链底开始向上遍历直到找到一个 j 满足 $val[j] < val[i]$ ，那么我们把 i 置为 j 的右儿子，把链底到 j 通过栈弹出的这条链作为 i 的左儿子。

显然该过程可以保证笛卡尔树的两个性质，由于每个点只会入栈一次和出栈一次，复杂度 $O(n)$

请注意：只有在权值两两不同时建出的笛卡尔树才唯一

```
class DescartesTree
{
public:
    int n, root;
    std::vector<int> lc, rc;
    DescartesTree(std::vector<int> &a) : n(a.size) - 1, lc(n + 1), rc(n + 1)
    {
        root = std::min_element(begin(a) + 1, end(a)) - begin(a);
        std::vector<int> stk(n + 1, 0);
        int top = 0;
        for (int i = 1; i <= n; ++i)
        {
            int k = top;
            while (k and a[stk[k]] > a[i])
                --k;
            if (k)
                rc[stk[k]] = i;
            if (k < top)
                lc[i] = stk[k + 1];
            stk[++k] = i;
            top = k;
        }
    }
};
```